LAPORAN PRATIKUM

Dosen pengampu : Dr.lr Septafiansyah Dwi Putra,S.T.,MT

Agus ambarwari,S.Pd.,M.Kom



Disusun oleh:

Nama : Mutia Qauna Hidayah

Npm : 25782048

Kelas : TRI 3B

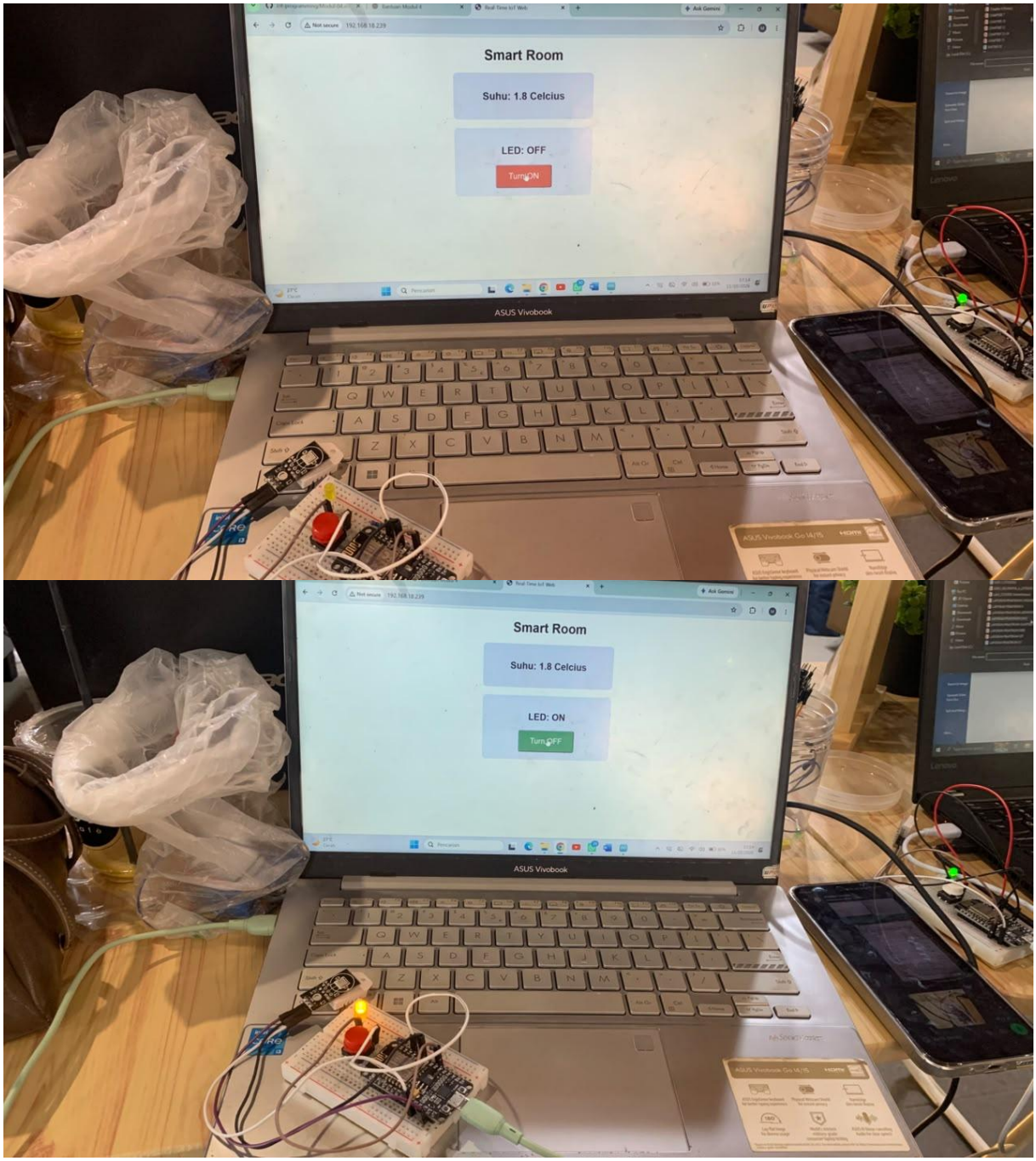
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG TAHUN 2026

4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik



Penjelasan Singkat Kode Khusus:

- `ws.cleanupClients()`: (Berada di siklus `loop()`). Fungsi penting dari pustaka *Asynchronous* ini bertugas membersihkan dan menutup sesi (*resource memori*) dari klien/*browser* yang sudah terputus atau menutup tab layarnya. Tanpa baris ini, memori RAM ESP8266 akan penuh (*memory leak*) dan berujung pada

Crash/Restart jika terlalu banyak perangkat keluar-masuk membuka halaman *Dashboard*.

- `pinMode(buttonPin, INPUT)` dan `if (buttonState == HIGH)`: Mempertahankan konfigurasi resistor *Pull-Down* eksternal 10k Ohm ke jalur GND (seperti pada Modul 1). Saat tombol dilepas, pin D2 membaca logika **LOW** (0V). Saat tombol ditekan, arus 3.3V mengalir sehingga terbaca logika **HIGH** yang memicu pembalikan status (*toggle*) LED secara instan.
- `if ((millis() - lastTime) > 3000)`: Pemrograman *non-blocking*. Alih-alih membuat CPU tidur selama 3 detik menggunakan `delay(3000)` (yang akan membekukan respons WebSocket), kita menggunakan selisih penghitung jam internal CPU (`millis()`). Dengan demikian, CPU dapat terus siaga mengawasi sentuhan tombol fisik dan interaksi jaringan internet dengan kecepatan tinggi.
- `var dataObj = JSON.parse(event.data)`: Sintaks JavaScript yang membedah teks kaku kiriman C++ berformat `{"suhu": "30", "led": "1"}` menjadi struktur data objek dinamis (*Dictionary*) untuk memperbarui elemen desain HTML dengan instan (*DOM Manipulation*).
- `ws.textAll()`: Perintah penyiaran (*broadcasting*) ampuh yang menembakkan/menyebarkan data secara seketika ke *seluruh* perangkat *browser* pengunjung (laptop, HP) secara bersamaan.

4.6 Latihan

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

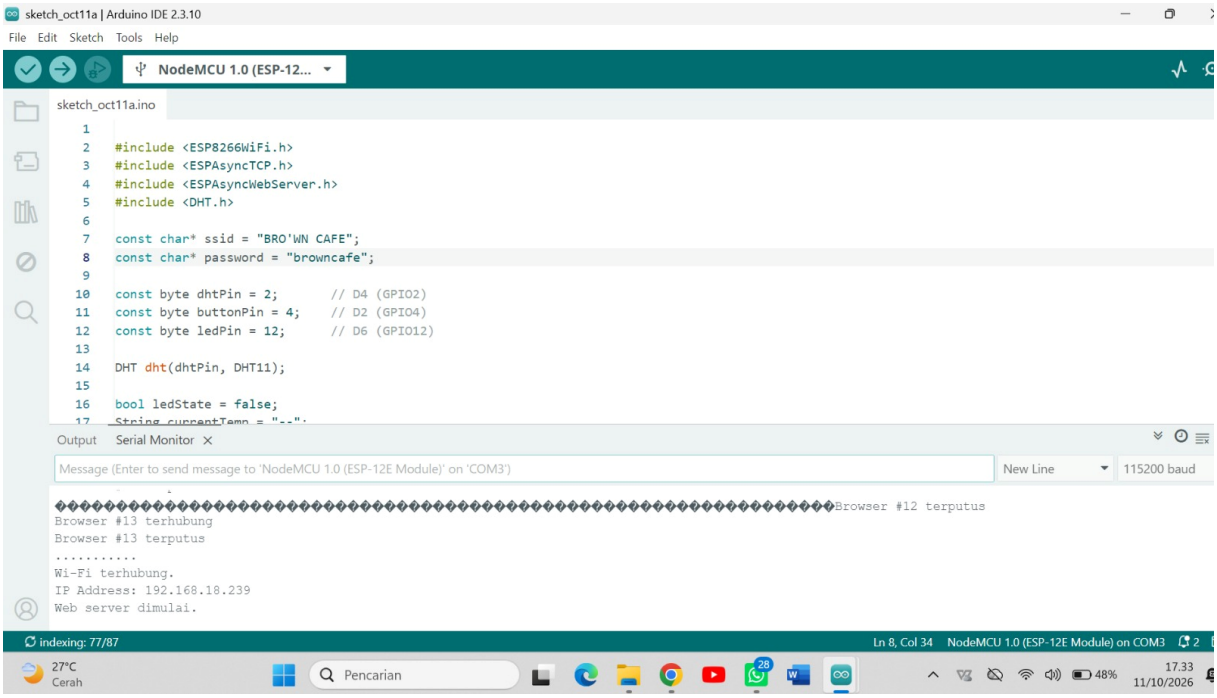
Jawab: Pada latihan 1, dilakukan pembuatan web server pada NodeMCU ESP8266 menggunakan WebSocket. WebSocket memungkinkan komunikasi dua arah antara perangkat dan browser secara real-time, sehingga data dapat dikirim tanpa harus memuat ulang halaman web.

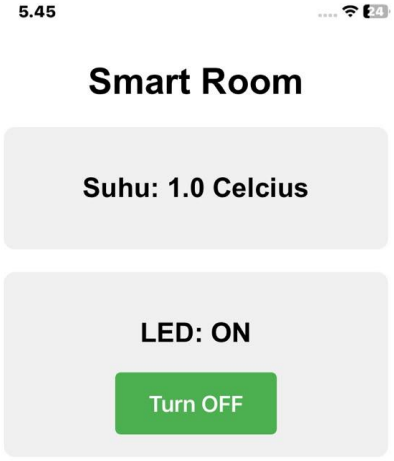
- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

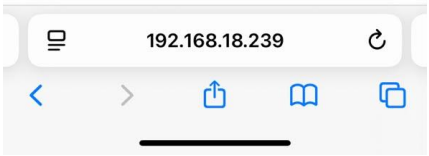


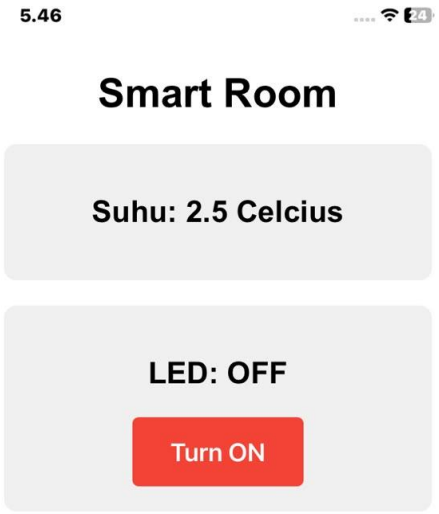
dilakukan pengembangan sistem dengan menambahkan fitur kendali LED melalui halaman web. LED dihubungkan ke pin digital NodeMCU, kemudian dikendalikan menggunakan tombol yang tersedia pada antarmuka web. Ketika tombol ditekan, browser mengirimkan pesan melalui WebSocket ke NodeMCU. Pesan tersebut diproses oleh program untuk mengubah kondisi LED, yaitu menyala atau mati. Status LED kemudian dikirim kembali ke browser sehingga pengguna dapat mengetahui kondisi perangkat. Praktikum ini menunjukkan bahwa perangkat keras dapat dikendalikan dari jarak jauh melalui halaman web selama perangkat dan browser terhubung ke jaringan yang sesuai.

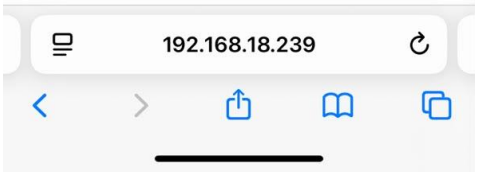
- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok `// Merombak UI Tombol & Teks sesuai status hardware`. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?


sketch_oct11a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
sketch_oct11a.ino
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>
const char* ssid = "BRO'WN CAFE";
const char* password = "browncafe";
const byte dhtPin = 2; // D4 (GPIO2)
const byte buttonPin = 4; // D2 (GPIO4)
const byte ledPin = 12; // D6 (GPIO12)
DHT dht(dhtPin, DHT11);
bool ledState = false;
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line
115200 baud
Browser #12 terputus
Browser #13 terhubung
Browser #13 terputus
.....
Wi-Fi terhubung.
IP Address: 192.168.18.239
Web server dimulai.
indexing: 77/87
Ln 8, Col 34 NodeMCU 1.0 (ESP-12E Module) on COM3
27°C Cerah
Pencarian
48%
17.33 11/10/2026


5.45
Smart Room
Suhu: 1.0 Celcius
LED: ON
Turn OFF


192.168.18.239



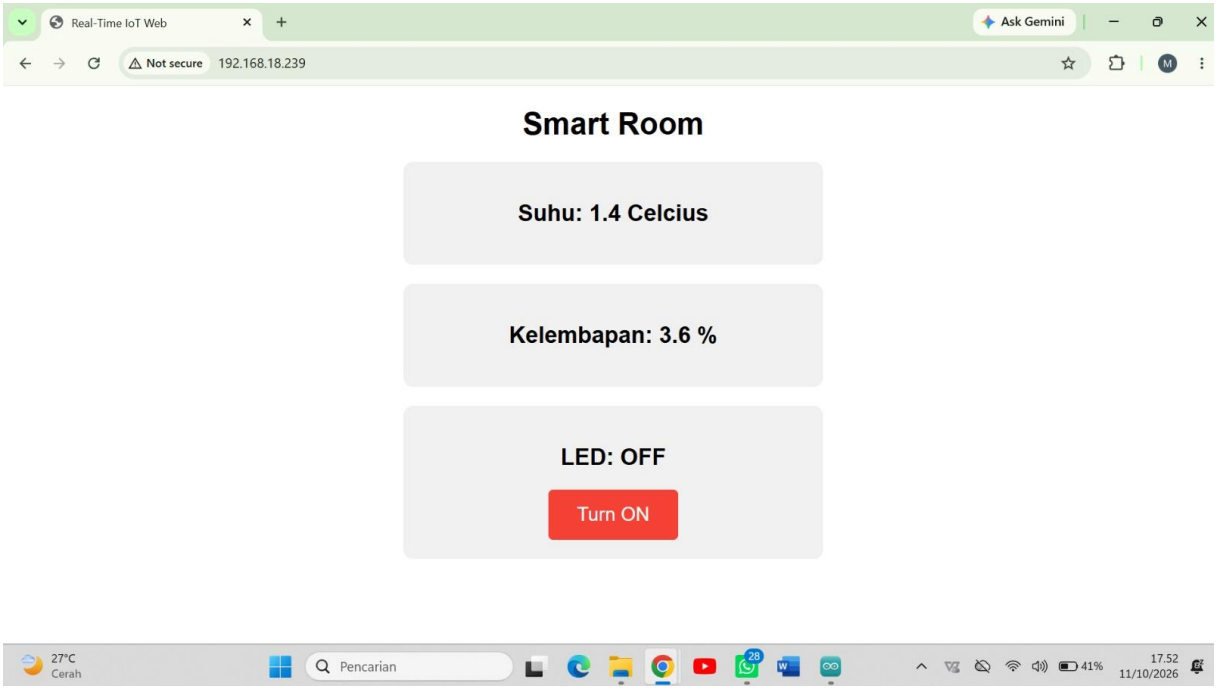


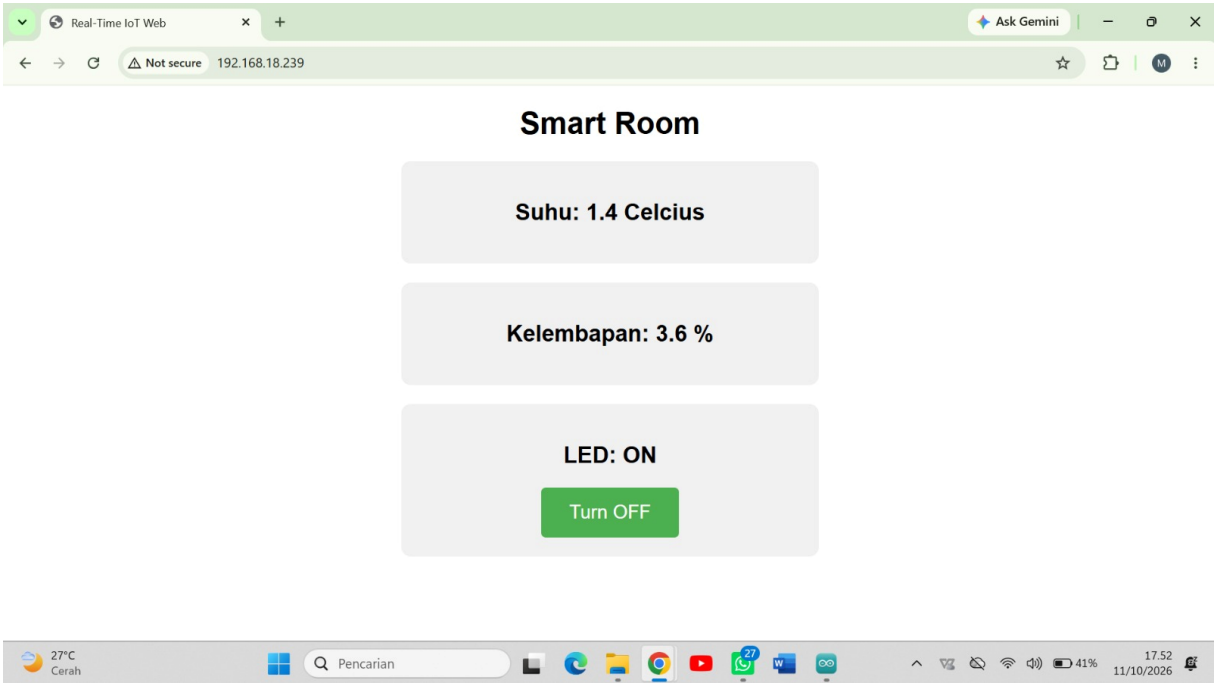
sistem dikembangkan dengan menambahkan tombol fisik (push button) sebagai alternatif untuk mengendalikan LED. LED dapat dinyalakan atau dimatikan melalui tombol fisik maupun tombol pada halaman web. Ketika tombol fisik ditekan, NodeMCU akan membaca perubahan sinyal pada pin input dan memperbarui status LED. Perubahan tersebut kemudian dikirim melalui WebSocket agar tampilan pada browser ikut berubah secara otomatis. Sebaliknya, ketika tombol pada halaman web ditekan, perintah dikirim ke NodeMCU untuk mengubah status LED dan memperbarui tampilan web. Program juga menggunakan teknik debouncing untuk mengurangi pembacaan ganda akibat perubahan sinyal tombol. Dengan sistem ini, kendali LED melalui perangkat fisik dan web dapat berjalan secara bersamaan dengan pembaruan status secara real-time.

- **Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
    i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam

paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).

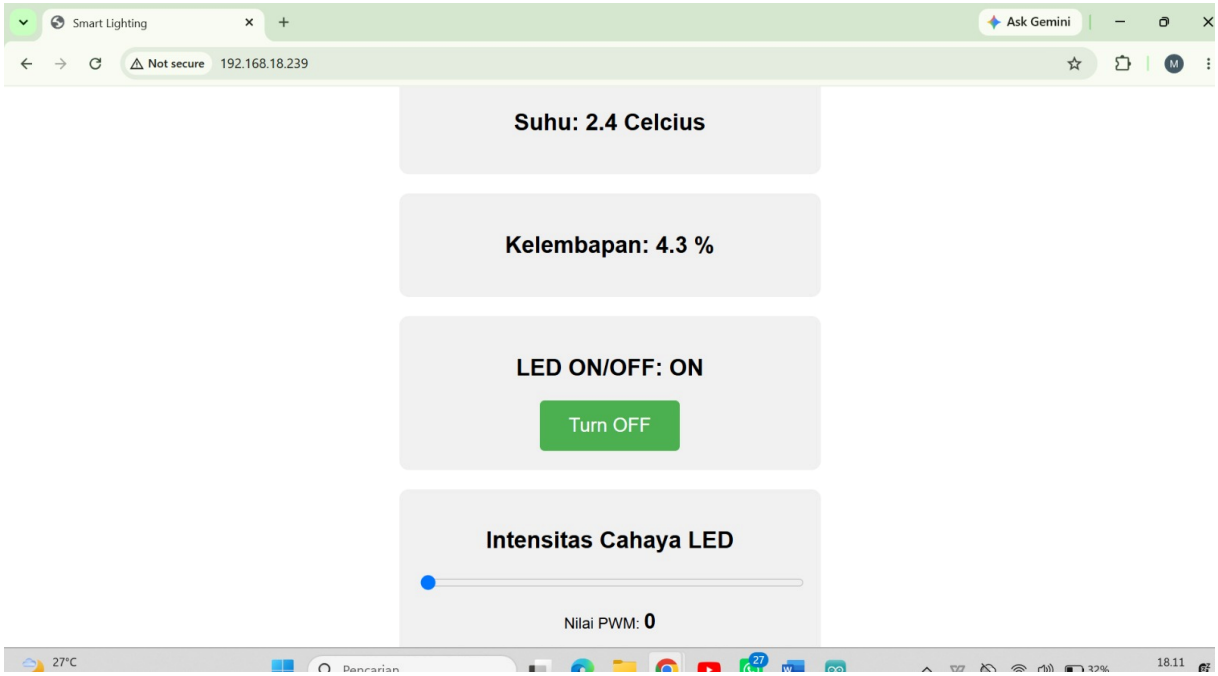
ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

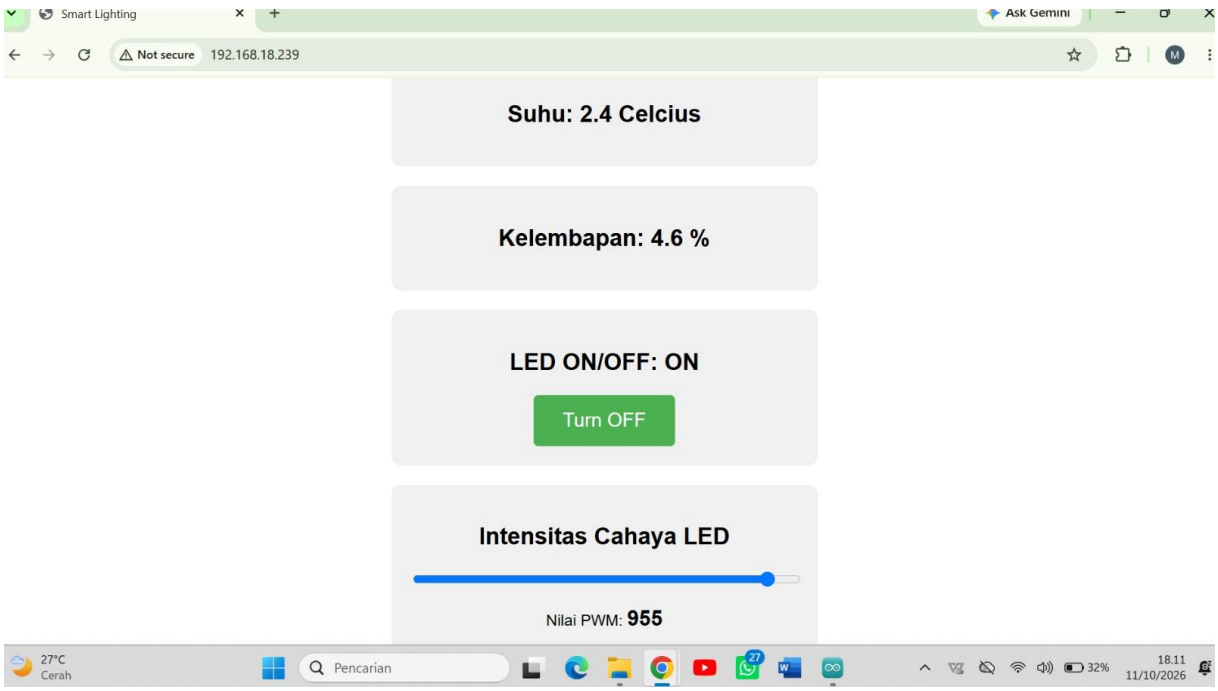


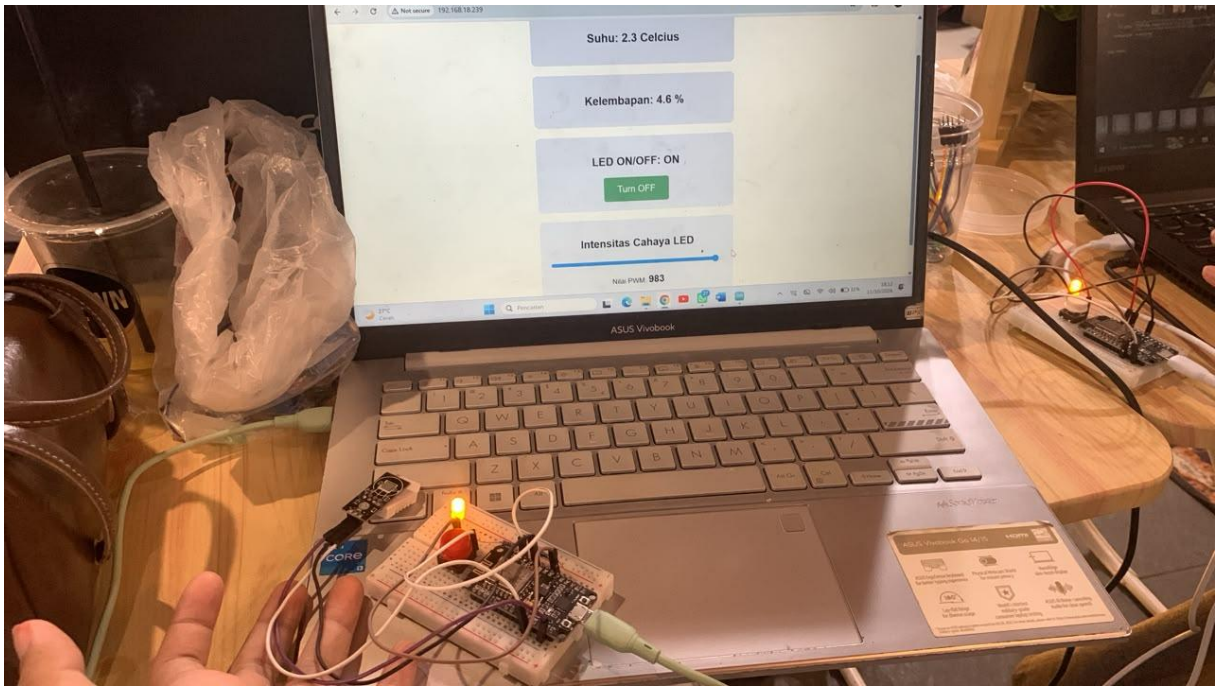


program dikembangkan dengan menambahkan fitur pembacaan kelembapan udara menggunakan sensor DHT11. Sebelumnya, halaman web hanya menampilkan data suhu, kemudian dimodifikasi agar dapat menampilkan suhu dan kelembapan sekaligus. Data suhu dibaca menggunakan fungsi `dht.readTemperature()`, sedangkan kelembapan dibaca menggunakan fungsi `dht.readHumidity()`. Hasil pembacaan disimpan dalam variabel, lalu dikirim ke browser dalam format JSON melalui WebSocket. Pada sisi HTML dan JavaScript, ditambahkan kartu kelembapan untuk menampilkan nilai sensor yang diterima dari NodeMCU. Pembacaan sensor dilakukan secara berkala menggunakan fungsi `millis()` dengan interval tiga detik, sehingga program tidak perlu berhenti menggunakan `delay(3000)` saat menunggu pembacaan berikutnya. Dengan penambahan fitur ini, pengguna dapat memantau kondisi suhu dan kelembapan ruangan secara real-time melalui halaman web.

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*







Pada tugas ini, sistem Smart Room dikembangkan dengan menambahkan fitur pengaturan intensitas cahaya LED menggunakan slider pada halaman web. Berbeda dengan kendali digital yang hanya memiliki kondisi menyala dan mati, metode PWM (Pulse Width Modulation) memungkinkan kecerahan LED diatur secara bertahap. Slider HTML dibuat menggunakan elemen `input type="range"` dengan rentang nilai 0 sampai 1023. Ketika slider digeser, JavaScript mengirimkan nilai tersebut ke NodeMCU melalui WebSocket dalam format pesan, misalnya `pwm,512`. Pada sisi program C++, pesan diperiksa dan nilai PWM dipisahkan untuk mendapatkan angka yang akan digunakan sebagai pengaturan kecerahan LED. Selanjutnya, fungsi `analogWrite()` digunakan untuk mengatur keluaran PWM pada pin D1 (GPIO 5). Nilai 0 membuat LED mati, sedangkan nilai 1023 menghasilkan tingkat kecerahan maksimum. Pada tugas ini, fitur kendali LED ON/OFF dan pembacaan sensor DHT11 tetap dipertahankan agar dapat berjalan bersamaan dengan pengaturan kecerahan LED. Hasilnya, pengguna dapat mengatur intensitas cahaya dari HP atau laptop melalui halaman web tanpa harus mengubah rangkaian secara manual.